SMART CALCULATOR

Complete Product Requirements Document (PRD) & Engineering Specification

Comprehensive Product Requirements, Vision Pipeline & Codebase Blueprint

Application Name:	Smart Calculator (Point. Snap. Solved.)
Android Package Name:	com.aistudio.smartcalc.wzqvxy
Current Version:	Version 1.8 (versionCode: 10)
Architecture Framework:	100% Kotlin | Jetpack Compose M3 | Clean MVVM | Coroutines Flow
Vision AI Models:	Google Gemini Flash Vision API (gemini-2.5-flash / gemini-1.5-flash)
Local Math Engine:	MathExpressionEvaluator (BODMAS/PEMDAS, Roman Numerals, Equations)
Min SDK / Target SDK:	Min SDK 24 (Android 7.0) | Target SDK 36 (Android 16)

1. Product Overview & Strategic Vision

Smart Calculator is a production-grade Android utility engineered to solve one of the most frustrating daily math bottlenecks: manually transcribing numbers, itemized bills, tilted handwritten notes, and algebraic equations into traditional calculator screens.

With the 'Point. Snap. Solved.' interaction paradigm, the user captures any physical receipt, whiteboard note, student homework sheet, or rough grocery list. The system leverages an on-device image pre-processing pipeline coupled with Google Gemini Vision AI to recognize complex handwriting with 100% precision. It computes results in milliseconds and provides an interactive breakdown with step-by-step audit trails and an ultra-smooth manual keypad.

CORE VALUE PROPOSITION

Zero-latency handwritten math recognition with 100% accuracy, seamless intermediate step breakdown, foolproof grand total selection, and a b

1.1 Target Personas & Use Cases

- **Shoppers & Kirana Stores:** Instant total calculation of handwritten grocery chits, rough bills, and item prices with Indian numbering formats.
- **Students & Educators:** Verifying multi-step arithmetic, parentheses balancing, Roman numeral problems (XIV * 5), and linear algebraic equations.
- **Professionals & Accountants:** Cross-checking printed invoice item lists, expense slips, and zig-zag sequential calculations without manual keying.

2. System Architecture & Technology Stack

The application adheres to clean Modern Android Architecture (MVVM) with unidirectional data flow (UDF), 100% Kotlin, and Jetpack Compose Material 3 styling:

Layer	Technologies / Libraries	Purpose
UI & Presentation	Jetpack Compose, Material 3, Dynamic Color	Declarative, fluid, edge-to-edge adaptive user interface
Asynchronous Flow	Kotlin Coroutines, StateFlow, Lifecycle-aware	Non-blocking background calculations, instant UI updates
Camera Subsystem	CameraX (View, Lifecycle, Camera2)	Hardware-accelerated viewfinder, tap-to-focus, flash toggle
Image Processing	Android Canvas, ColorMatrix, Matrix, Unsharp	Contrast stretching, stroke sharpening, 768px normalization
Vision AI Engine	Gemini 2.5/1.5 Flash Vision REST API	High-accuracy handwriting OCR & math understanding
Local Math Evaluator	MathExpressionEvaluator (RPN, BigDecimal)	Zero-latency in-memory calculation, Roman numerals & BODMAS
Local Persistence	SharedPreferences, StateSaveable	Remembering theme preferences and active calculator inputs

3. High-Accuracy Vision OCR & Pre-Processing Pipeline

To achieve 100% recognition on slanted, messy, low-contrast, or unaligned handwriting, raw captured images pass through a multi-stage pre-processing pipeline prior to cloud inference:

- **GPU ColorMatrix Normalization:** Applies dynamic contrast stretching (1.45x) and brightness calibration to separate faint pencil/pen strokes from paper background.
- **Stroke Edge Sharpening:** Enhances stroke borders to clearly distinguish easily confused handwritten characters (e.g., '1' vs '7', '0' vs '6', '+' vs 'x', '/' vs '-').
- **Payload Dimension Optimization:** Downscales high-resolution camera photos to a max boundary of 768px. This preserves digit clarity while shrinking JPEG payload to ~40KB, reducing upload transmission latency to sub-80ms.
- **Rotational Normalization:** Detects orientation EXIF metadata and user manual rotation (90 deg increments), ensuring handwriting is perfectly horizontal before analysis.

4. Grand Total Selection Logic & Fallback Parsing Engine

A critical challenge in multi-line receipt and calculation OCR is avoiding false positives where intermediate steps (e.g., '405') are mistaken for the final Grand Total (e.g., '420'). Smart Calculator implements a resilient two-tier resolution strategy:

- **Prompt-Level Strict Mandate:** The Gemini Vision prompt explicitly instructs the model to reserve 'grand_total' strictly for the ultimate bottom line / total sum, classifying all prior lines as itemized steps.
- **Deterministic Client Fallback:** In parseGeminiResponseWithFallbacks(), the app verifies if grand_total matches an intermediate step. If multiple calculation lines exist and a keyword like 'Total', 'Net', or 'Sum' appears in the final item, the system automatically promotes the last calculated line item as the authoritative Grand Total.
- **Heuristic Raw Text Recovery:** If the JSON response contains irregularities or stray marks, fallbackParseRawText() extracts lines, tokenizes operands, and passes them to the local in-memory math engine, guaranteeing zero 'no math found' errors on valid inputs.

5. In-Memory BODMAS / PEMDAS Math Engine

The MathExpressionEvaluator singleton handles instant, zero-latency calculation across standard arithmetic and specialized notations:

- **Standard BODMAS/PEMDAS:** Evaluates brackets, exponents, division, multiplication, addition, and subtraction with exact precedence using the Shunting-Yard algorithm and Reverse Polish Notation (RPN).
- **Arbitrary Precision:** Calculates all operations using java.math.BigDecimal with 10 decimal digits of precision to prevent floating-point rounding errors.
- **Roman Numeral Engine:** Converts Roman numbers (I, V, X, L, C, D, M) to Arabic integers (e.g., 'XIV * 5' -> 70) and formats explanations.
- **Contextual Percentages & Discounts:** Recognizes commercial patterns such as '500 - 10%' -> '500 - (500 * 10 / 100) = 450' and '1000 + 18%' -> 1180.

- **Single-Variable Linear Equations:** Detects algebraic expressions such as '2x + 6 = 20' or 'x - 5 = 10' and computes the value of x.
- **Zig-Zag Sequential Chains:** Parses linked calculations containing '->', '=>', or '->' arrows (e.g., '10 + 20 -> 30 * 2 -> 60').

6. UI & Interactive Breakdown Screen

The results screen provides transparency and user control:

- **Hero Result Card:** Prominently displays the Final Grand Total with high-contrast Material 3 typography and a vibrant 'Final Answer' chip badge.
- **Horizontal Step Filter Chips:** Allows users to switch between individual calculation steps. Defaults to selecting the Grand Total line.
- **Step-by-Step Audit Trail:** Displays each recognized step with its original handwritten text alongside the parsed algebraic expression and individual result.
- **Live Inline Formula Editing:** Tapping any recognized step opens an inline editor. Modifying digits or signs recalculates the step and downstream totals instantaneously.
- **One-Tap Actions:** Provides 'Copy Result', 'Scan Again', and 'Open in Calculator' buttons to transfer formulas into the manual keypad without retyping.

7. Ultra-Smooth Native Manual Keypad Specification

The manual keypad is engineered for buttery-smooth responsiveness, tactile feedback, and ergonomic performance:

- **Micro-Tactile Haptic Feedback:** Replaced sluggish vibration motor queues with lightweight TextHandleMove haptic feedback, preventing latency during fast multi-digit typing.
- **Hardware-Accelerated Ripples:** Utilizes Material 3 Surface with combinedClickable, executing touch feedback directly on the RenderThread with zero dropped frames.
- **Post-Calculation State Handling:** Pressing a digit immediately after '=' starts a fresh calculation, while pressing an operator continues smoothly with the calculated answer.
- **Continuous Expression Scrolling:** Horizontal scrolling display automatically tracks and scrolls to the newest typed characters for long formulas.
- **Smart Parentheses & Operators:** Automatically balances '(' and ')' parentheses and seamlessly replaces duplicate consecutive operators.
- **Fast Clear Gesture:** Long-pressing the DEL backspace button immediately clears the entire calculation line.
- **Cross-Tab State Preservation:** Synchronized with MainViewModel, preserving in-progress calculations when toggling between Scan and Calculator tabs.

8. Crop, Rotate & Image Workbench

Before OCR execution, users have full control over the captured frame:

- **Free Aspect Ratio Cropping:** Interactive 4-corner draggable handles allow arbitrary framing of vertical receipts, slanted notebook pages, or tight single equations.
- **90-Degree Lossless Rotation:** Allows rotating images to orient inverted or landscape handwriting.
- **Bounding Box Normalization:** Crop coordinates are normalized between 0.0f and 1.0f, ensuring accurate pixel extraction on any screen density.

9. Core Data Models & State Machine

Key immutable domain models driving the application:

Model / State	Primary Fields	Role in Architecture
MathOcrResult	foundMath, grandTotal, expressions, note	Parsed vision output representing full image math
DetectedMathItem	originalText, expression, calculatedResult, isTotal	Individual arithmetic step within a multi-line list
ScanUiState	Idle, CameraPreview, CropPreview, Processing, Success, Error	Sealed class modeling the complete scan state machine
ActiveTab	SCAN, CALCULATOR	Root navigation state for top-level app switching
ThemeMode	SYSTEM, LIGHT, DARK	User visual appearance preference state

10. Edge Cases, Error Handling & Recovery Matrix

Scenario	Potential Failure	System Defense & Recovery Action
Messy / Faint Pencil	Low contrast OCR failure	ColorMatrix 1.45x contrast stretch + kernel edge sharpening
Blank / Non-Math Photo	False hallucination	foundMath flag set to false, displays clean friendly guidance card
Zero / Negative Denominator	Arithmetic crash / Infinity	Evaluator catches ArithmeticException and displays 'Division by zero'
Intermediate Step Confusion	Displays 405 instead of 420	Grand Total logic enforces selection of last line item or Total badge
Network Timeout / Offline	Hangs indefinitely	12s connect / 20s read timeouts with candidate model failover
Fast Rapid Typing	Button sticking / lag	RenderThread ripples, non-blocking micro-haptics, pre-allocated key rows

11. Quality Assurance, Unit Tests & Build Runbook

All business logic, parsing heuristics, and image transformations are validated via automated Robolectric local JVM test suites:

- **Robolectric Test Runner:** Runs under sdk = [36] with Android context mocking (:app:testDebugUnitTest).
- **Grand Total Disambiguation Test:** Validates that multi-step lists (e.g. '120 + 35 -> 155; 250 -> 250; Total: 405 + 15 = 420') choose 420 over 405.
- **BODMAS Evaluation Tests:** Validates operator precedence, Roman numerals ('XIV * 5 = 70'), implicit multiplications ('2(3) = 6'), and vertical columns.
- **Lossless Crop Tests:** Verifies that normalized crop rectangles yield exact expected pixel dimensions without memory leaks.
- **Build Verification Command:** Run 'gradle :app:testDebugUnitTest' followed by 'compile_applet' to verify pristine zero-warning compilation.